

Mean-Variance Optimization

Mathematical optimization is a very difficult problem in general, particularly when we are dealing with complex objectives and constraints. However, **convex optimization** problems are a well-understood class of problems, which happen to be incredibly useful for finance. A convex problem has the following form:

$$\begin{array}{ll} \underset{\mathbf{x}}{\text{minimise}} & f(\mathbf{x}) \\ \text{subject to} & g_i(\mathbf{x}) \leq 0, i = 1, \dots, m \\ & A\mathbf{x} = b, \end{array}$$

where $\mathbf{x} \in \mathbb{R}^n$, and $f(\mathbf{x}), g_i(\mathbf{x})$ are convex functions. [\[1\]](#)

Fortunately, portfolio optimization problems (with standard objectives and constraints) are convex. This allows us to immediately apply the vast body of theory as well as the refined solving routines – accordingly, the main difficulty is inputting our specific problem into a solver.

PyPortfolioOpt aims to do the hard work for you, allowing for one-liners like `ef.min_volatility()` to generate a portfolio that minimises the volatility, while at the same time allowing for more complex problems to be built up from modular units. This is all possible thanks to [cvxpy](#), the *fantastic* python-embedded modelling language for convex optimization upon which PyPortfolioOpt's efficient frontier functionality lies.

Tip

You can find complete examples in the relevant cookbook [recipe](#).

Structure

As shown in the definition of a convex problem, there are essentially two things we need to specify: the optimization objective, and the optimization constraints. For example, the classic portfolio optimization problem is to **minimise risk** subject to a **return constraint** (i.e the portfolio must return more than a certain amount). From an implementation perspective, however, there is not much difference between an objective and a constraint. Consider a similar ~~problem which~~ is to **maximize return** subject to a **risk constraint** – now, the role of risk and return is swapped.  [stable](#) ▼

To that end, PyPortfolioOpt defines an `objective_functions` module that contains objective functions (which can also act as constraints, as we have just seen). The actual optimization occurs in the `efficient_frontier.EfficientFrontier` class. This class provides straightforward methods for optimising different objectives (all documented below).

However, PyPortfolioOpt was designed so that you can easily add new constraints or objective terms to an existing problem. For example, adding a regularisation objective (explained below) to a minimum volatility objective is as simple as:

```
ef = EfficientFrontier(expected_returns, cov_matrix) # setup
ef.add_objective(objective_functions.L2_reg) # add a secondary objective
ef.min_volatility() # find the portfolio that minimises volatility and L2_reg
```

! Tip

If you would like to plot the efficient frontier, take a look at the [Plotting](#) module.

Basic Usage

The `efficient_frontier` module houses the `EfficientFrontier` class and its descendants, which generate optimal portfolios for various possible objective functions and parameters.

`class pypf.efficient_frontier.EfficientFrontier(expected_returns, cov_matrix, weight_bounds=(0, 1), solver=None, verbose=False, solver_options=None)` [\[source\]](#)

An `EfficientFrontier` object (inheriting from `BaseConvexOptimizer`) contains multiple optimization methods that can be called (corresponding to different objective functions) with various parameters. Note: a new `EfficientFrontier` object should be instantiated if you want to make any change to objectives/constraints/bounds/parameters.

Instance variables:

- Inputs:
 - `n_assets` - int
 - `tickers` - str list
 - `bounds` - float tuple OR (float tuple) list
 - `cov_matrix` - np.ndarray
 - `expected_returns` - np.ndarray
 - `solver` - str
 - `solver_options` - {str: str} dict

- Output: `weights` - `np.ndarray`

Public methods:

- `min_volatility()` optimizes for minimum volatility
- `max_sharpe()` optimizes for maximal Sharpe ratio (a.k.a the tangency portfolio)
- `max_quadratic_utility()` maximises the quadratic utility, given some risk aversion.
- `efficient_risk()` maximises return for a given target risk
- `efficient_return()` minimises risk for a given target return
- `add_objective()` adds a (convex) objective to the optimization problem
- `add_constraint()` adds a constraint to the optimization problem
- `convex_objective()` solves for a generic convex objective with linear constraints
- `portfolio_performance()` calculates the expected return, volatility and Sharpe ratio for the optimized portfolio.
- `set_weights()` creates `self.weights` (`np.ndarray`) from a weights dict
- `clean_weights()` rounds the weights and clips near-zeros.
- `save_weights_to_file()` saves the weights to csv, json, or txt.

`__init__(expected_returns, cov_matrix, weight_bounds=(0, 1), solver=None, verbose=False, solver_options=None)` [\[source\]](#)

Parameters:

- **expected_returns** (*pd.Series, list, np.ndarray*) – expected returns for each asset. Can be None if optimising for volatility only (but not recommended).
- **cov_matrix** (*pd.DataFrame or np.array*) – covariance of returns for each asset. This **must** be positive semidefnite, otherwise optimization will fail.
- **weight_bounds** (*tuple OR tuple list, optional*) – minimum and maximum weight of each asset OR single min/max pair if all identical, defaults to (0, 1). Must be changed to (-1, 1) for portfolios with shorting.
- **solver** (*str*) – name of solver. list available solvers with:
`cvxpy.installed_solvers()`
- **verbose** (*bool, optional*) – whether performance and debugging info should be printed, defaults to False
- **solver_options** (*dict, optional*) – parameters for the given solver

Raises:

- **TypeError** – if `expected_returns` is not a series, list or array
- **TypeError** – if `cov_matrix` is not a dataframe or array

! Note

As of v0.5.0, you can pass a collection (list or tuple) of (min, max) pairs representing different bounds for different assets.

! Tip

If you want to generate short-only portfolios, there is a quick hack. Multiply your expected returns by -1, then optimize a long-only portfolio.

min_volatility() [\[source\]](#)

Minimise volatility.

Returns: asset weights for the volatility-minimising portfolio

Return type: OrderedDict

max_sharpe(risk_free_rate=0.02) [\[source\]](#)

Maximise the Sharpe Ratio. The result is also referred to as the tangency portfolio, as it is the portfolio for which the capital market line is tangent to the efficient frontier.

This is a convex optimization problem after making a certain variable substitution. See [Cornuejols and Tutuncu \(2006\)](#) for more.

Parameters: **risk_free_rate** (*float, optional*) – risk-free rate of borrowing/lending, defaults to 0.02. The period of the risk-free rate should correspond to the frequency of expected returns.

Raises: **ValueError** – if `risk_free_rate` is non-numeric

Returns: asset weights for the Sharpe-maximising portfolio

Return type: OrderedDict

! Caution

Because `max_sharpe()` makes a variable substitution, additional objectives may not work as intended.

max_quadratic_utility(risk_aversion=1, market_neutral=False) [\[source\]](#)

Maximise the given quadratic utility, i.e:

$$\max_w w^T \mu - \frac{\delta}{2} w^T \Sigma w$$

- Parameters:**
- **risk_aversion** (*positive float*) – risk aversion parameter (must be greater than 0), defaults to 1
 - **market_neutral** – whether the portfolio should be market neutral (weights sum to zero), defaults to False. Requires negative lower weight bound.
 - **market_neutral** – bool, optional

Returns: asset weights for the maximum-utility portfolio

Return type: OrderedDict

! Note

`pypfopt.black_litterman` provides a method for calculating the market-implied risk-aversion parameter, which gives a useful estimate in the absence of other information!

efficient_risk(*target_volatility, market_neutral=False*) [\[source\]](#)

Maximise return for a target risk. The resulting portfolio will have a volatility less than the target (but not guaranteed to be equal).

- Parameters:**
- **target_volatility** (*float*) – the desired maximum volatility of the resulting portfolio.
 - **market_neutral** – whether the portfolio should be market neutral (weights sum to zero), defaults to False. Requires negative lower weight bound.
 - **market_neutral** – bool, optional

- Raises:**
- **ValueError** – if `target_volatility` is not a positive float
 - **ValueError** – if no portfolio can be found with volatility equal to `target_volatility`
 - **ValueError** – if `risk_free_rate` is non-numeric

Returns: asset weights for the efficient risk portfolio

Return type: OrderedDict

! Caution

If you pass an unreasonable target into `efficient_risk()` or `efficient_re` optimizer will fail silently and return weird weights. *Caveat emptor* applies.  **stable** ▼

efficient_return(*target_return*, *market_neutral=False*) [\[source\]](#)

Calculate the 'Markowitz portfolio', minimising volatility for a given target return.

Parameters:

- **target_return** (*float*) – the desired return of the resulting portfolio.
- **market_neutral** (*bool, optional*) – whether the portfolio should be market neutral (weights sum to zero), defaults to False. Requires negative lower weight bound.

Raises:

- **ValueError** – if `target_return` is not a positive float
- **ValueError** – if no portfolio can be found with return equal to `target_return`

Returns: asset weights for the Markowitz portfolio

Return type: OrderedDict

portfolio_performance(*verbose=False*, *risk_free_rate=0.02*) [\[source\]](#)

After optimising, calculate (and optionally print) the performance of the optimal portfolio. Currently calculates expected return, volatility, and the Sharpe ratio.

Parameters:

- **verbose** (*bool, optional*) – whether performance should be printed, defaults to False
- **risk_free_rate** (*float, optional*) – risk-free rate of borrowing/lending, defaults to 0.02. The period of the risk-free rate should correspond to the frequency of expected returns.

Raises: **ValueError** – if weights have not been calculated yet

Returns: expected return, volatility, Sharpe ratio.

Return type: (float, float, float)

! Tip

If you would like to use the `portfolio_performance` function independently of any optimizer (e.g for debugging purposes), you can use:

```
from pypfopt import base_optimizer

base_optimizer.portfolio_performance(
    weights, expected_returns, cov_matrix, verbose=True, risk_free_rate=0.02
)
```

! Note

PyPortfolioOpt defers to cvxpy's default choice of solver. If you would like to explicitly choose the solver, simply pass the optional `solver = "ECOS"` kwarg to the constructor. You can choose from any of the [supported solvers](#), and pass in solver params via `solver_options` (a `dict`).

Adding objectives and constraints

EfficientFrontier inherits from the BaseConvexOptimizer class. In particular, the functions to add constraints and objectives are documented below:

class `pypfopt.base_optimizer.BaseConvexOptimizer`

`BaseConvexOptimizer.add_constraint(new_constraint)`

Add a new constraint to the optimization problem. This constraint must satisfy DCP rules, i.e be either a linear equality constraint or convex inequality constraint.

Examples:

```
ef.add_constraint(lambda x : x[0] == 0.02)
ef.add_constraint(lambda x : x >= 0.01)
ef.add_constraint(lambda x: x <= np.array([0.01, 0.08, ..., 0.5]))
```

Parameters: `new_constraint` – the constraint to be added

`BaseConvexOptimizer.add_sector_constraints(sector_mapper, sector_lower, sector_upper)`

Adds constraints on the sum of weights of different groups of assets. Most commonly, these will be sector constraints e.g portfolio's exposure to tech must be less than x%:

```

sector_mapper = {
    "GOOG": "tech",
    "FB": "tech",
    "XOM": "Oil/Gas",
    "RRC": "Oil/Gas",
    "MA": "Financials",
    "JPM": "Financials",
}

sector_lower = {"tech": 0.1} # at least 10% to tech
sector_upper = {
    "tech": 0.4, # less than 40% tech
    "Oil/Gas": 0.1 # less than 10% oil and gas
}

```

- Parameters:
- **sector_mapper** (*{str: str} dict*) – dict that maps tickers to sectors
 - **sector_lower** (*{str: float} dict*) – lower bounds for each sector
 - **sector_upper** (*{str: float} dict*) – upper bounds for each sector

BaseConvexOptimizer.add_objective(new_objective, **kwargs)

Add a new term into the objective function. This term must be convex, and built from cvxpy atomic functions.

Example:

```

def L1_norm(w, k=1):
    return k * cp.norm(w, 1)

ef.add_objective(L1_norm, k=2)

```

- Parameters:
- **new_objective** (*cp.Expression (i.e function of cp.Variable)*) – the objective to be added

Objective functions

The `objective_functions` module provides optimization objectives, including the actual objective functions called by the `EfficientFrontier` object's optimization methods. These methods are primarily designed for internal use during optimization and each requires a different signature (which is why they have not been factored into a class). For obvious reasons, any function must accept `weights` as an argument, and must also have at least one of `expected_returns` or `cov_matrix`.



The objective functions either compute the objective given a numpy array of weights, or they return a cvxpy *expression* when weights are a `cp.Variable`. In this way, the same objective function can be used both internally for optimization and externally for computing the objective given weights. `_objective_value()` automatically chooses between the two behaviours.

`objective_functions` defaults to objectives for minimisation. In the cases of objectives that clearly should be maximised (e.g Sharpe Ratio, portfolio return), the objective function actually returns the negative quantity, since minimising the negative is equivalent to maximising the positive. This behaviour is controlled by the `negative=True` optional argument.

Currently implemented:

- Portfolio variance (i.e square of volatility)
- Portfolio return
- Sharpe ratio
- L2 regularisation (minimising this reduces nonzero weights)
- Quadratic utility
- Transaction cost model (a simple one)
- Ex-ante (squared) tracking error
- Ex-post (squared) tracking error

`pypfopt.objective_functions.L2_reg(w, gamma=1)` [\[source\]](#)

L2 regularisation, i.e $\gamma ||w||^2$, to increase the number of nonzero weights.

Example:

```
ef = EfficientFrontier(mu, S)
ef.add_objective(objective_functions.L2_reg, gamma=2)
ef.min_volatility()
```

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **gamma** (*float, optional*) – L2 regularisation parameter, defaults to 1. Increase if you want more non-negligible weights

Returns: value of the objective function OR objective function expression

Return type: float OR *cp.Expression*

pypfopt.objective_functions.ex_ante_tracking_error(*w*, *cov_matrix*, *benchmark_weights*)

[\[source\]](#)

Calculate the (square of) the ex-ante Tracking Error, i.e $(w - w_b)^T \Sigma (w - w_b)$.

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **cov_matrix** (*np.ndarray*) – covariance matrix
- **benchmark_weights** (*np.ndarray*) – asset weights in the benchmark

Returns: value of the objective function OR objective function expression

Return type: float OR *cp.Expression*

pypfopt.objective_functions.ex_post_tracking_error(*w*, *historic_returns*, *benchmark_returns*)

[\[source\]](#)

Calculate the (square of) the ex-post Tracking Error, i.e $Var(r - r_b)$.

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **historic_returns** (*np.ndarray*) – historic asset returns
- **benchmark_returns** (*pd.Series* or *np.ndarray*) – historic benchmark returns

Returns: value of the objective function OR objective function expression

Return type: float OR *cp.Expression*

pypfopt.objective_functions.portfolio_return(*w*, *expected_returns*, *negative=True*) [\[source\]](#)

Calculate the (negative) mean return of a portfolio

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **expected_returns** (*np.ndarray*) – expected return of each asset
- **negative** (*boolean*) – whether quantity should be made negative (so we can minimise)

Returns: negative mean return

Return type: float

 [stable](#) ▼

pypfopt.objective_functions.portfolio_variance(*w*, *cov_matrix*) [\[source\]](#)

Calculate the total portfolio variance (i.e square volatility).

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **cov_matrix** (*np.ndarray*) – covariance matrix

Returns: value of the objective function OR objective function expression

Return type: float OR *cp.Expression*

pypfopt.objective_functions.quadratic_utility(*w*, *expected_returns*, *cov_matrix*, *risk_aversion*, *negative=True*) [\[source\]](#)

Quadratic utility function, i.e $\mu - \frac{1}{2} \delta w^T \Sigma w$.

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **expected_returns** (*np.ndarray*) – expected return of each asset
- **cov_matrix** (*np.ndarray*) – covariance matrix
- **risk_aversion** (*float*) – risk aversion coefficient. Increase to reduce risk.
- **negative** (*boolean*) – whether quantity should be made negative (so we can minimise).

Returns: value of the objective function OR objective function expression

Return type: float OR *cp.Expression*

pypfopt.objective_functions.sharpe_ratio(*w*, *expected_returns*, *cov_matrix*, *risk_free_rate=0.02*, *negative=True*) [\[source\]](#)

Calculate the (negative) Sharpe ratio of a portfolio

Parameters:

- **w** (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- **expected_returns** (*np.ndarray*) – expected return of each asset
- **cov_matrix** (*np.ndarray*) – covariance matrix
- **risk_free_rate** (*float, optional*) – risk-free rate of borrowing/lending, defaults to 0.02. The period of the risk-free rate should correspond to the frequency of expected returns.
- **negative** (*boolean*) – whether quantity should be made negative (so we can minimise)

Returns: (negative) Sharpe ratio

Return type: float

pypfopt.objective_functions.transaction_cost(*w*, *w_prev*, *k*=0.001) [\[source\]](#)

A very simple transaction cost model: sum all the weight changes and multiply by a given fraction (default to 10bps). This simulates a fixed percentage commission from your broker.

Parameters:

- *w* (*np.ndarray* OR *cp.Variable*) – asset weights in the portfolio
- *w_prev* (*np.ndarray*) – previous weights
- *k* (*float*) – fractional cost per unit weight exchanged

Returns: value of the objective function OR objective function expression

Return type: float OR *cp.Expression*

More on L2 Regularisation

As has been discussed in the [User Guide](#), mean-variance optimization often results in many weights being negligible, i.e the efficient portfolio does not end up including most of the assets. This is expected behaviour, but it may be undesirable if you need a certain number of assets in your portfolio.

In order to coerce the mean-variance optimizer to produce more non-negligible weights, we add what can be thought of as a “small weights penalty” to all of the objective functions, parameterised by γ ([gamma](#)). Considering the minimum variance objective for instance, we have:

$$\underset{w}{\text{minimise}} \{w^T \Sigma w\} \longrightarrow \underset{w}{\text{minimise}} \{w^T \Sigma w + \gamma w^T w\}$$

Note that $w^T w$ is the same as the sum of squared weights (I didn't write this explicitly to reduce confusion caused by Σ denoting both the covariance matrix and the summation operator). This term reduces the number of negligible weights, because it has a minimum value when all weights are equally distributed, and maximum value in the limiting case where the entire portfolio is allocated to one asset. I refer to it as **L2 regularisation** because it has exactly the same form as the L2 regularisation term in machine learning, though a slightly different purpose (in ML it is used to keep weights small while here it is used to make them larger).

 **Note**

In practice, γ must be tuned to achieve the level of regularisation that you want. However, if the universe of assets is small (less than 20 assets), then `gamma=1` is a good starting point. For larger universes, or if you want more non-negligible weights in the final portfolio, increase `gamma`.

References

- [1] Boyd, S.; Vandenberghe, L. (2004). [Convex Optimization](#).